DATA COUCH

Real-Time Streaming & Lakehouse Pipelines

Introduction to Redpanda Connect

DAY-2

Introduction to Redpanda Connect

What is Redpanda Connect?

Redpanda Connect is a data streaming service for building scalable, high-performance data pipelines that drive real-time analytics and actionable business insights.

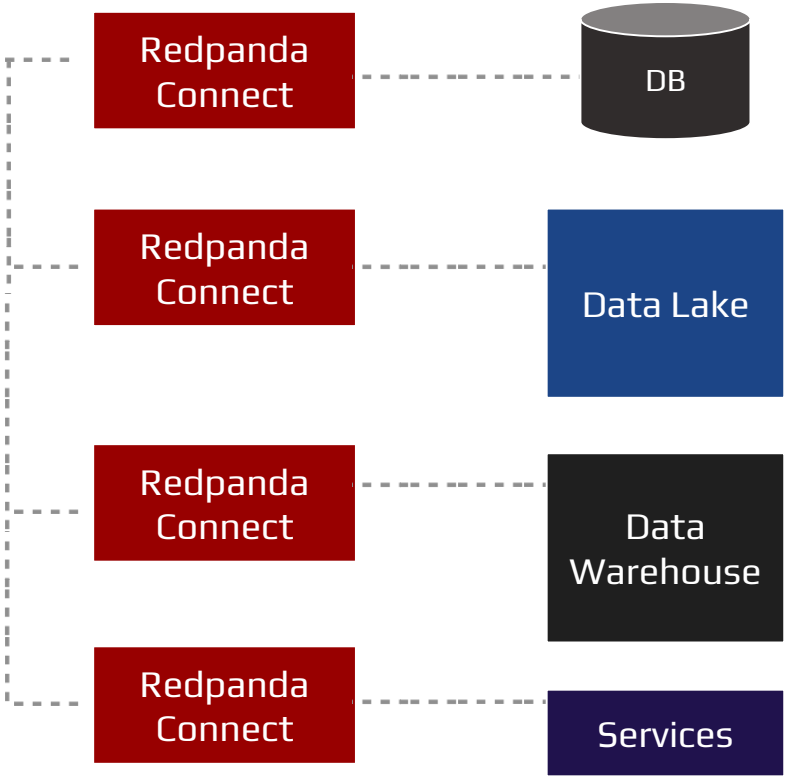
Integrate data across systems with hundreds of prebuilt connectors, change data capture (CDC) capabilities, and YAML-configurable pipelines.

Redpanda Connect = plug-and-play pipelines for databases, streams, warehouses, and cloud storage without writing custom code.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

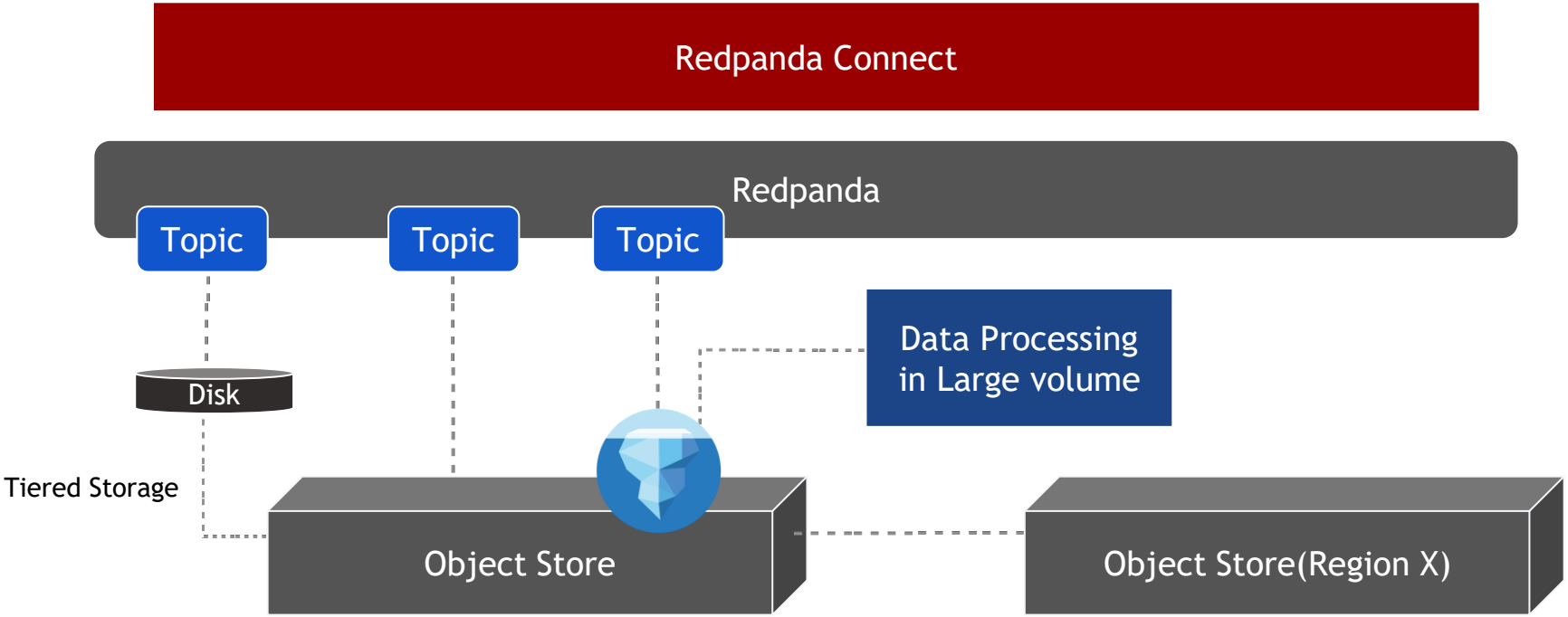
Architecture and philosophy

Redpanda Connect is a framework for moving data in and out of Redpanda without writing custom code.



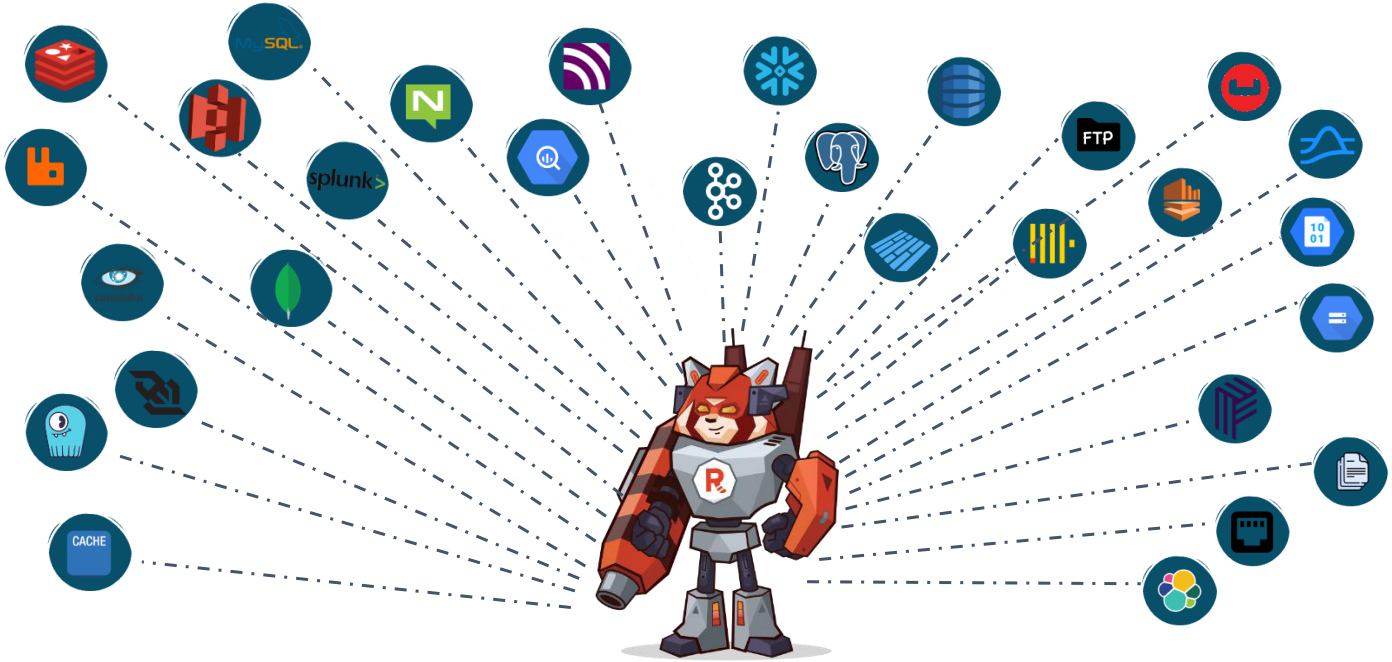
Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda One



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda Connect - Streaming ETL



Transformations | Enrichments | Rate limits

Validation | Filtering | Hydration

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Architecture and philosophy

Single Binary,
no more runtime dependency

Simple YAML,
no more Java classes names

Payload agnostic,
Support structured or binary data

Redpanda Connect is

boring

Stateless and fast,
no more complex cluster setup

Out of Box Observability,
Tracing, Metrics, and Logs

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Architecture and philosophy

rpk connect **run** config.yaml

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Configuring Redpanda Connect

Input

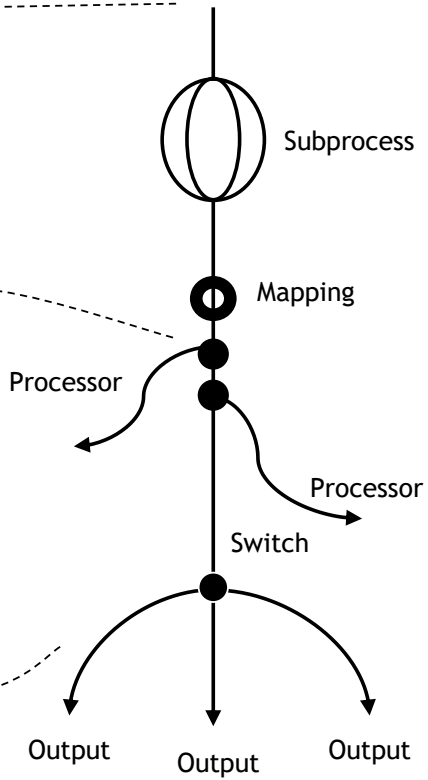
```
input:
  sftp:
    address: 123.45.678.90
    paths: ["upload/*.csv"]
    codec: lines
```

Processor

```
pipeline:
  processors:
    - bloblang: |
        root.id = this.msg_id
        root.body = this.content
    - branch:
        processors:
          -aws_lambda:
              function: calc_interest
        resultmap: root.meta,interest = meta
```

Output

```
output:
  gcp_pubsub:
    project: "demo"
    topic: "assets"
    max_in_flight: 64
```



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Intuitive Declarative Configuration

Error handling
- retries
- drop
- reject
- DLQ

Bloblang
- jq and jsonpath
- commonly used functions
- frequently used methods
- advance mapping features

Buffer
- batching
- system window

Cache
- deduplication
- data joins
- enrichment

Observability
- metrics
- tracing
- healthchecks

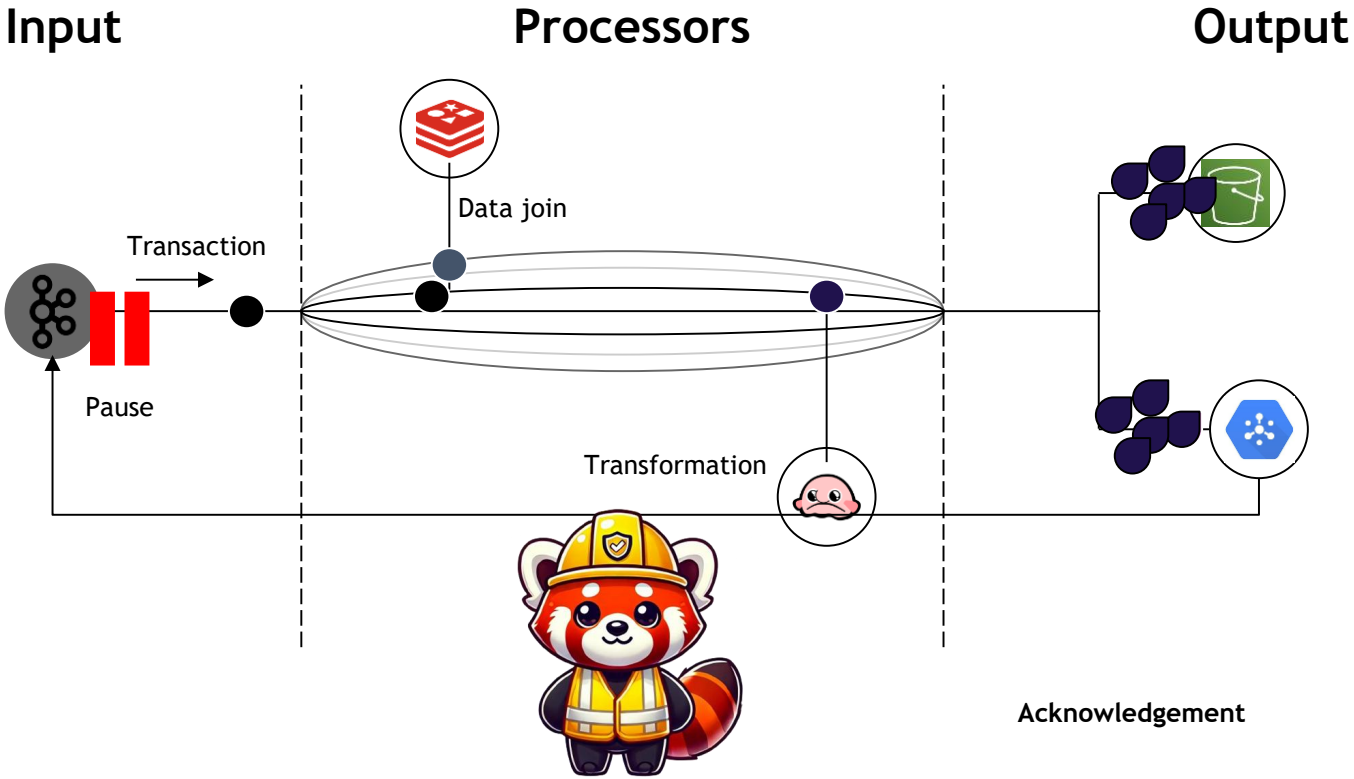
Input

Processor

Output

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Crash Resilient, Delivery Guarantee - It is Safe



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Just run it!

create

lint

rpk connect **run** config.yaml

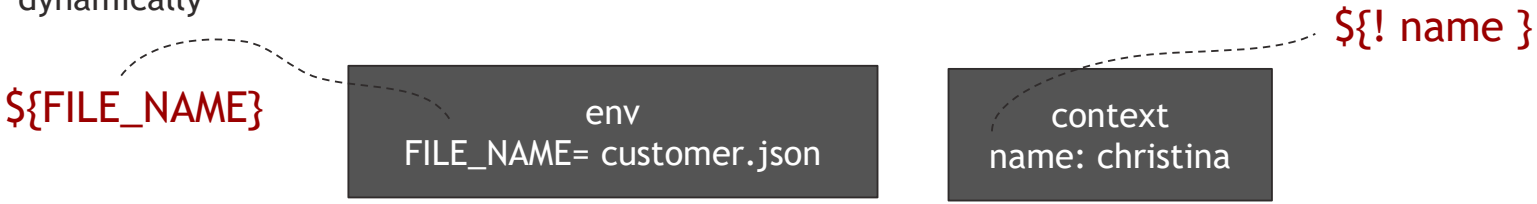
streams

test

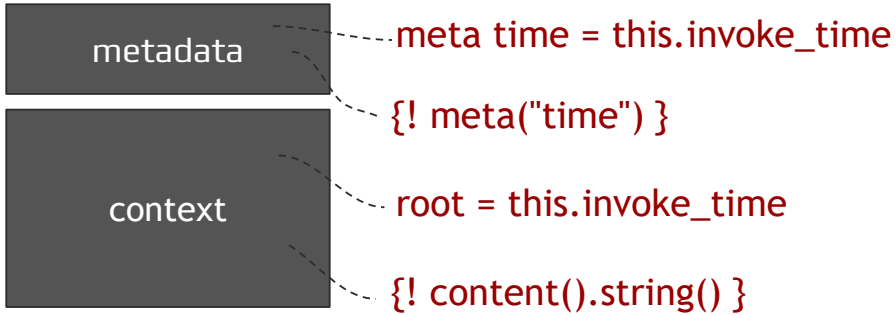
Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Input and Output

- Supports string interpolation and environment variables so it does configuration dynamically



- Each message has raw contents and metadata, which is a map of key/value pairs representing an arbitrary amount of complementary data



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Building Pipelines

YAML-based declarative pipeline configs

Redpanda Connect pipelines are configured in a YAML file that consists of a number of root sections

```yaml
input:
  kafka:
    addresses: [ TODO ]
    topics: [ foo, bar ]
    consumer_group: foogroup

pipeline:
  processors:
  - mapping: |
      root.message = this
      root.meta.link_count = this.links.length()

output:
  aws_s3:
    bucket: TODO
    path: '${! meta("kafka_topic") }/${! json("message.id") }.json'
```

This document introduces Redpanda Connect tooling to simplify the writing and management of complex configuration files.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Prebuilt Connectors

Redpanda Connect comes with a wide library of ready-to-use connectors so you don't have to build everything from scratch.

These connectors cover both:

- **sources** (where data comes from)
- **sinks** (where data goes).

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Prebuilt Connectors

Common Prebuilt Source Connectors

- **Databases** – PostgreSQL, MySQL, MongoDB, Oracle, SQL Server
- **Message Brokers** – Apache Kafka, RabbitMQ, NATS, MQTT
- **Cloud Storage** – AWS S3, GCP Storage, Azure Blob, MinIO
- **APIs & Streams** – HTTP, WebSockets, CDC (Change Data Capture), FileStream

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Prebuilt Connectors

Common Prebuilt Sink Connectors

- **Databases** – PostgreSQL, MySQL, ClickHouse, Snowflake, MongoDB
- **Cloud Storage** – AWS S3, GCS, Azure Blob, MinIO
- **Message Queues & Streams** – Kafka, Pulsar, RabbitMQ, NATS
- **Warehouses & Analytics** – BigQuery, Redshift, Snowflake, ElasticSearch, OpenSearch

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

In-flight Transformations with Bloblang

What is Bloblang?

A lightweight mapping language to reshape, enrich, filter, and route messages in streaming pipelines (used in Redpanda Connect/Benthos). Think: JSON-in → rules → JSON-out, at very high throughput.

When to use it

- *Clean/normalize events before sinking to DB/warehouse*
- *Enrich with constants/metadata/lookup results*
- *Mask/obfuscate PII*
- *Branch, drop, or reroute messages based on content*

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Mapping

Input document immutability:

Mapping creates a new object for assignments, keeping the original document immutable and always queryable during transformations.

Error handling:

Bloblang mappings can fail, in which case the message remains unchanged, errors are logged, and the message is flagged as having failed, allowing you to use standard processor error handling patterns.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Filtering and Sampling

Filtering events in Redpanda Connect is both easy and flexible, this cookbook demonstrates a few different types of filtering you can do. All of these examples make use of the mapping processor but shouldn't require any prior knowledge.

The basic filter

Dropping events with Bloblang is done by mapping the function deleted() to the root of the mapped document. To remove all events indiscriminately you can simply do:

```
pipeline:
  processors:
  - mapping: root = deleted()
```

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Sample events

Another type of filter we might want is a sampling filter, we can do that with a random number generator:

```
pipeline:
  processors:
  - mapping: |
      # Drop 50% of documents randomly
      root = if random_int() % 2 == 0 {
deleted() }
```

We can also do this in a deterministic way by hashing events and filtering by that hash value:

```
pipeline:
  processors:
  - mapping: |
      # Drop ~10% of documents deterministically (same docs filtered each run)
      root = if content().hash("xxhash64").slice(-8).number() % 10 == 0 {
         deleted()
      }
```

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Enrichment

Redpanda Connect enables real-time data enrichment by adding context, transforming formats, and integrating external sources like caches or AI systems.

How Enrichment Works in Redpanda Connect

- **Workflow Processors:** Perform multiple enrichments in parallel using dependency-aware processors.
- **Declarative Configuration:** Define pipelines and enrichment logic via simple YAML.
- **Integration:** Connect to databases, APIs, files, sinks, or AI models for transformations.
- **Reusable Code:** Apply JavaScript, Wasm, AWS Lambda, or Linux commands per message.
- **Benthos Integration:** Enables advanced mapping, filtering, and enrichment for complex analytics and AI use cases.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Conditional logic

Redpanda Connect uses processors, Bloblang mapping, and branching to enable dynamic, conditional pipeline processing.

How Conditional Logic Works in Redpanda Connect

- **Workflow Branches:** Execute pipeline stages based on specific criteria.
- **Bloblang Mapping:** Use if statements and match expressions for conditional mapping, including nested structures.
- **Filtering & Dropping:** Use deleted() to skip messages that don't meet conditions.
- **Conditional Routing:** Route outputs dynamically based on message content.
- **Practical Use:** Enables real-time filtering and selective enrichment, e.g., processing only relevant user activity or adding missing context.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

What is Apache Iceberg, and why does it matter?

Introducing Apache Iceberg

*Apache Iceberg is an **open table format** for huge analytic datasets in your data lake (like S3).*

*It is **not** a query engine (like Spark or Trino) and **not** a storage system (like S3). It is the **metadata layer** that sits between them, bringing **sanity, reliability, and performance** to your data lake.*

ICEBERG

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

What Is Apache Iceberg?

Apache Iceberg is an open table format designed for huge analytical datasets stored in data lakes like S3, ADLS, and GCS.

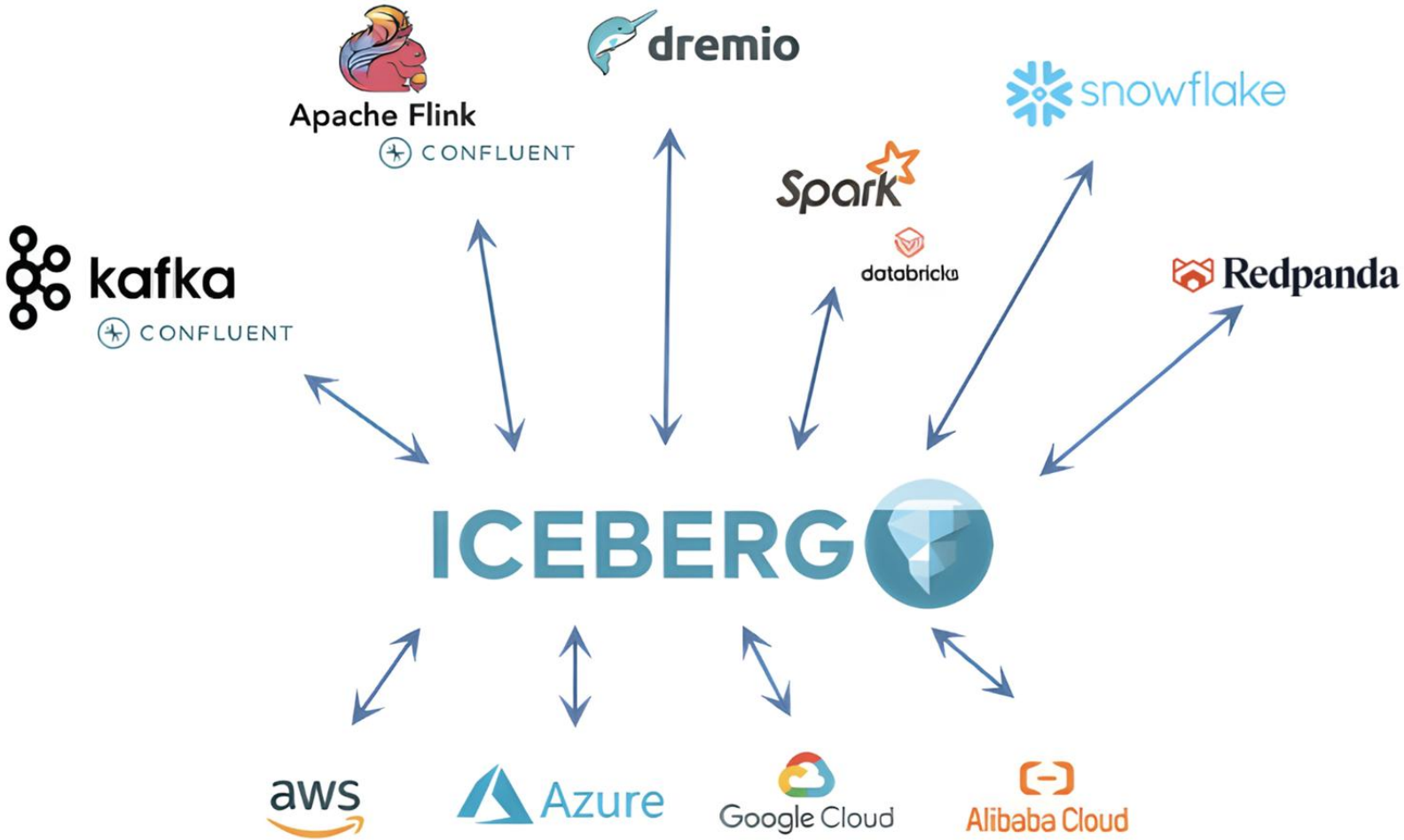
Iceberg *Is*

- *An open table format specification*
- *A metadata layer for data lakes*
- *Engine-agnostic (works with Spark, Trino, Flink)*
- *Designed for petabyte-scale analytics*

Iceberg *Is Not*

- *A storage system (like S3 or HDFS)*
- *A query engine (like Spark or Presto)*
- *A database management system*
- *A data processing framework*

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Why Apache Iceberg Matters

100%

ACID Compliance

Full transactional guarantees

100%

Schema Evolution

Safe, non-breaking changes

100%

Time Travel

Query any historical version

100%

Performance

Optimised query execution

100%

Engine Agnostic

Works with Spark, Trino, Flink

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Table Formats vs Traditional Lakes

Why Traditional Data Lakes Fail

Data lakes promised cheap, scalable storage for analytics. But without proper management, they quickly deteriorate into unusable data swamps.

No ACID Guarantees

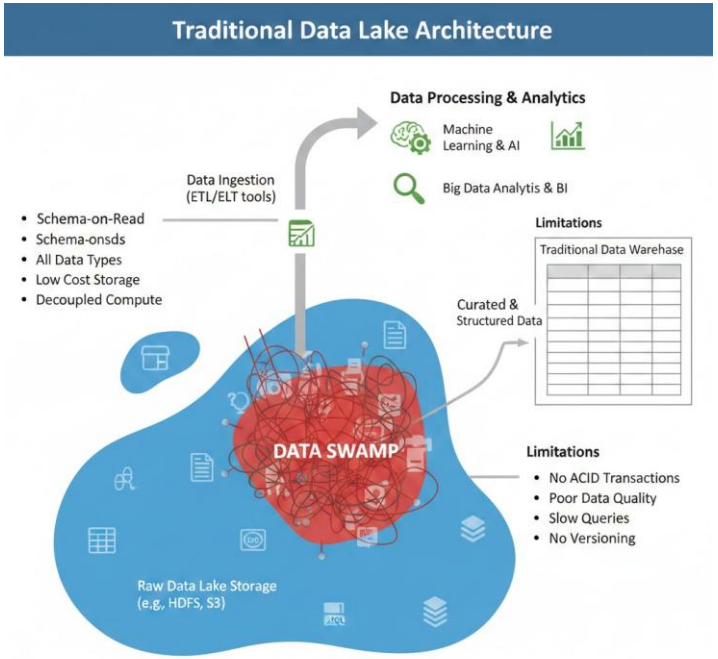
Partial writes create corrupted datasets. No transactional consistency across files.

Brittle Schema Management

Schema changes break existing queries. No safe evolution path for production tables.

Inefficient Query Performance

Queries scan thousands of unnecessary files, leading to slow execution and excessive costs.



Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

Data Files: Immutable Storage

Data files store the actual table content, typically in Parquet format (though ORC and Avro are also supported).

Key Characteristics

Immutable: Never updated in place

Versioned: Old files remain untouched

Efficient: Columnar format optimised for analytics

When data changes, new files are written whilst old files are preserved, enabling time travel and safe rollbacks.

Copyright © 2025 by DataCouch Pvt. Ltd.. All rights reserved.

THANK YOU